Student Workbook

One chapter per week: ideas, pictures, exercises and new words

Tip: How to use this workbook

Bring this workbook to every session. Read the chapter with your partner, try every exercise, and fill in the reflection at the end. It is fine to write in it, draw in it and make mistakes in it!

Contents

Week 1: Robots Need Instructions 2
Week 2: Loop-de-Loop 7
Week 3: When This Happens... 12
Week 4: Keeping Score 16
Week 5: Messages and My Blocks 20
Week 6: Review Studio and Checkpoint 25
Week 7: Hello, Python! 32
Week 8: Loops in Python 37
Week 9: Variables, Input and Choices 41
Week 10: Random Art and My Own Commands 45
Week 11: Project Studio 48
Week 12: Showcase and Celebration 51

Week 1 Level 1: Introduction

Robots Need Instructions

A program is a list of exact steps, done in order.

Our class agreements

Digital citizenship & safety: How we work together

1. **Be kind.** We laugh with each other, never at a mistake. Bugs are normal!
2. **Share the mouse.** The driver clicks, the navigator helps. We swap when the timer rings.
3. **Point, don't grab.** To help a partner, point at the screen and explain.
4. **Nicknames only.** No real full names, addresses, school names, birthdays or photos in our projects.
5. **Offline and private.** Scratch stays on this Mac. No accounts, no sharing online. If something asks for your name or a password, stop and get a grown-up.
6. **Look after your body.** Take stretch breaks and rest your eyes.

My coding nickname: _____ I agree: _____

What you will learn

- What an **algorithm** is, and why the order of steps matters.
- How to find your way around the Scratch app.
- How to snap blocks together and start them with the **green flag**.
- How to find and fix a bug when steps are in the wrong order.

The big idea

A computer is like a robot. It does **exactly** what you tell it, **in the order** you tell it. It never guesses what you meant.

A list of exact steps in order is called an **algorithm**. When a computer follows an algorithm, we call it a **program**. A mistake in a program is a **bug**. Finding and fixing it is called **debugging**.

Tip: Try it on paper

Number these steps 1 to 5 so a robot can make a jam sandwich.

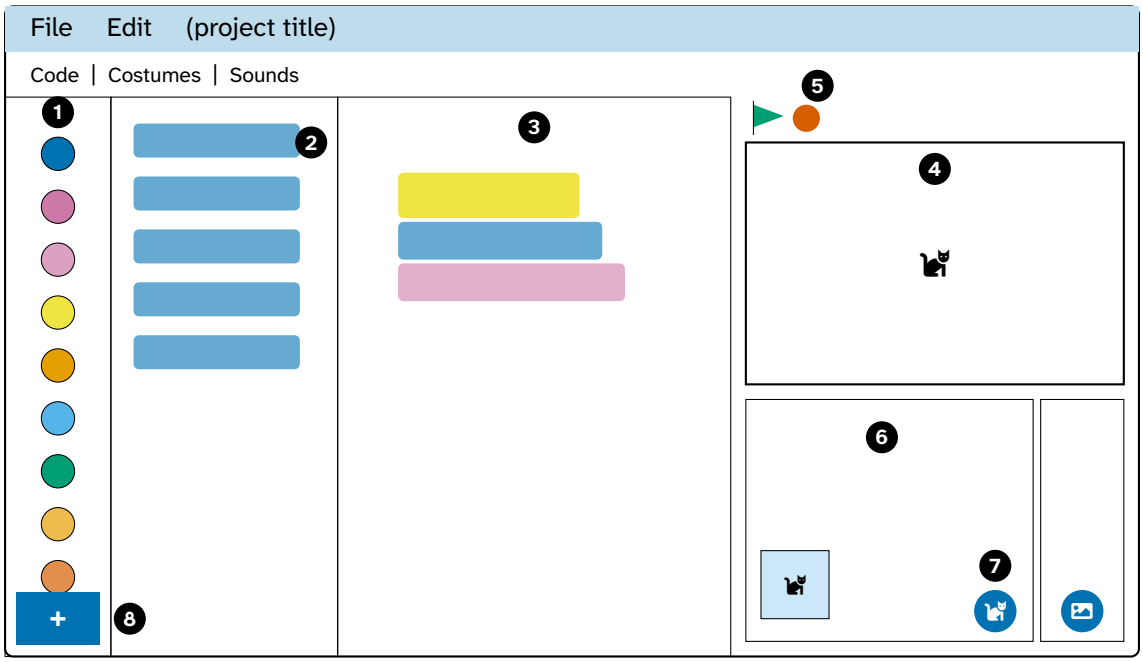
_____ Spread jam on one slice. _____ Put the other slice on top.

_____ Take two slices of bread. _____ Open the jam jar.

_____ Eat and enjoy!

Meet Scratch

Scratch is a coding app. You make programs by snapping blocks together, like puzzle pieces.



1 Block categories (colored circles: Motion, Looks, Sound, Events, ...)

2 Blocks you can drag

3 Code area: snap blocks here to make a **script**

4 Stage: where your sprites act

5 Green flag (start) and red stop sign

6 Sprite pane: all your sprites

7 Choose a Sprite button (and Choose a Backdrop, far right)

8 Add Extension button (we use it in week 2)

Exercise 1.1: Hello, Cat!

Make the cat say hello, walk and say bye.

1. **Events**: drag “when green flag clicked” into the code area.

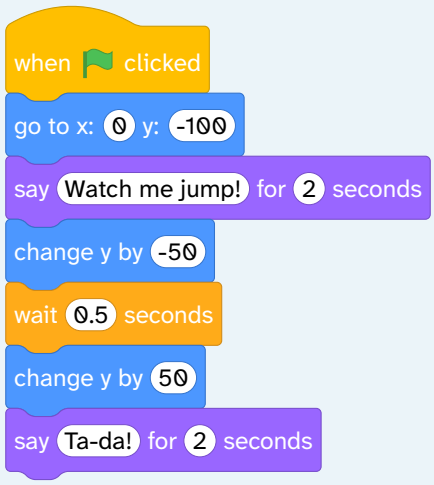
2. **Motion**: snap “go to x: 0 y: 0” under it. That is the middle of the stage.
3. **Looks**: snap “say Hello! for 2 seconds”. Type “Hi! I am” and your nickname.
4. **Motion**: snap “move 10 steps”. Change 10 to 100.
5. **Sound**: snap “play sound Meow until done”.
6. **Looks**: snap one more “say … for 2 seconds” and type “Bye!”.
7. Click the green flag. Save: **File** → **Save to your computer**, folder `Scratch Projects`.



Done when: the cat talks, walks, meows and says bye, in that order, every time you click the flag.

Exercise 1.2: Mixed-Up Jump (bug hunt)

The cat should say “Watch me jump!”, jump **up**, come back down to the ground and say “Ta-da!”. Build this script **exactly** as it is, bug and all. (**change y by** is in **Motion**: y is up and down.)



Click the flag. What did the cat do?

Which two blocks are in the wrong order?

Now drag them into the right order and test again. Did the cat jump up? ☐ yes

Tip: Debugging steps

1. What did I **expect**? 2. What **happened**? 3. Read the blocks one at a time with your finger. 4. Change **one** thing and test again.

Exercise 1.3: My Sprite Show

Make your own little show.

1. Click **Choose a Sprite** (the cat-face button, bottom right) and pick any sprite you like.
2. Click **Choose a Backdrop** (far bottom right) and pick a place for your show.
3. Plan 5 steps below, then build them under “when green flag clicked”.

My show plan (one step per line):

Done when: your show starts with the green flag and the steps happen in your planned order.

Vocabulary box

algorithm A list of exact steps, in order, that solves a problem.

program An algorithm that a computer can follow.

sequence Steps that happen one after another, in order.

bug / debugging A mistake in a program / finding and fixing it.

sprite A character or object on the Scratch stage, like the cat.

stage The area where your sprites act.

script A stack of blocks snapped together.

green flag The button that starts your program.

driver / navigator The driver clicks; the navigator reads, checks and helps.

Reflection

What surprised you about giving instructions to a robot?

One thing I want to make in Scratch:

Week 2 Level 1: Introduction

Loop-de-Loop

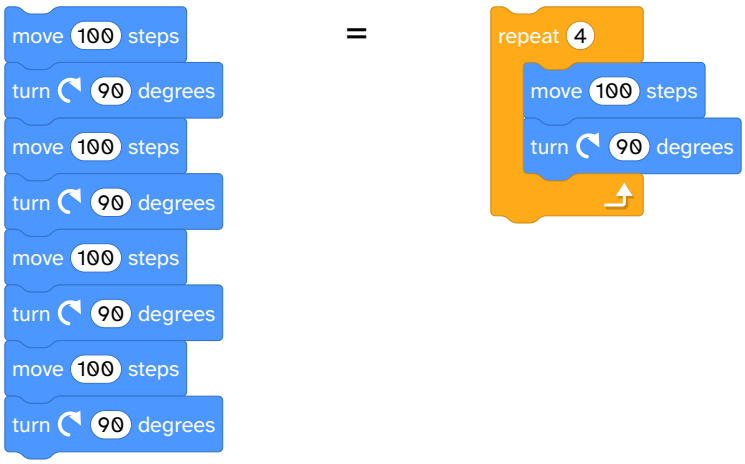
Loops repeat steps so we don't have to.

What you will learn

- How **repeat** and **forever** loops save you from copying blocks.
- How to make a sprite walk by switching **costumes**.
- How to draw with the **Pen** and make shapes with the rule **turn = 360 ÷ sides**.

The big idea

Look at these two scripts. They draw the same square! The left one copies the same two blocks four times. The right one uses a **loop**.

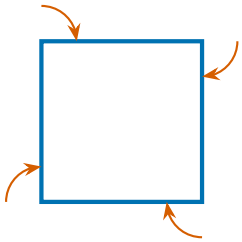


- **repeat 4** runs the blocks inside it 4 times, then the program carries on.
- **forever** runs the blocks inside it again and again until you click the red stop sign.

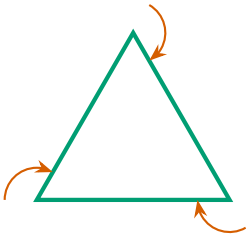
How much to turn?

When you walk all the way around a shape, you turn all the way around once. A whole turn is **360 degrees**. So each corner turn is:

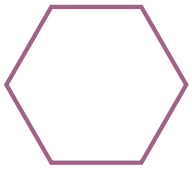
turn = 360 ÷ number of sides



Square: 4 turns
360 ÷ 4 = **90**



Triangle: 3 turns
360 ÷ 3 = **120**



Hexagon: 6 turns
360 ÷ 6 = **60**

The arrows show the turn at each corner. Big arrows = big turns.

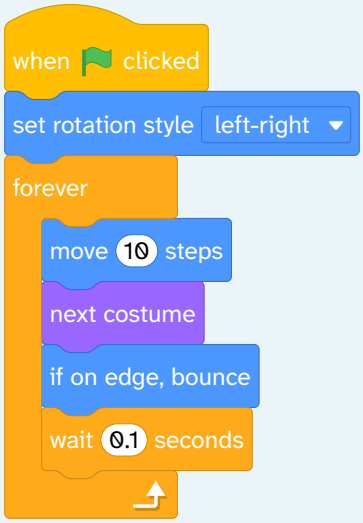
Fill in the table. (Tip: you can let Scratch divide! Put the green "( ) / ( )" block from **Operators** into the turn slot.)

Shape	Sides	Repeat	Turn (degrees)
Triangle	3	3	120
Square	4		
Pentagon	5		
Hexagon	6		
Octagon	8		

Exercise 2.1: Walking Cat

Make the cat walk back and forth forever.

1. Start a new project (**File** → **New**). Click the **Costumes** tab: the cat has 2 costumes. Flipping between them looks like walking! Click **Code** to go back.
2. Build this script. "set rotation style" keeps the cat from walking upside down.
3. Click the flag. Click the red stop sign to stop.

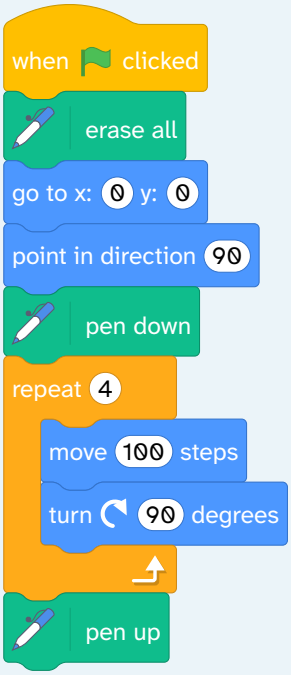


Try it: what happens if you take out the “wait” block?

Exercise 2.2: Polygon Party

Draw shapes with the Pen.

1. Click the blue **Add Extension** button (bottom left) and choose **Pen**.
2. Make the cat small: in the sprite pane, set **Size** to 40.
3. Build the square script. Click the flag. Does the square close?
4. Now change the numbers to draw a triangle, a hexagon and a pentagon. Use your table!



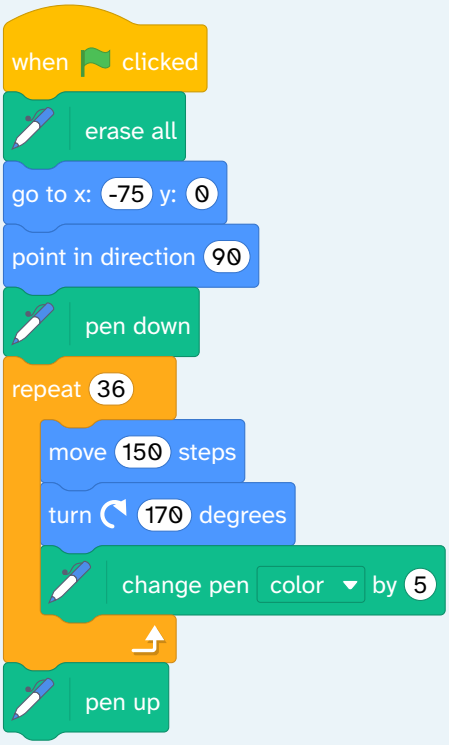
Bug hunt! Leo wanted a triangle. He used repeat **3**, move 100, turn **60**.

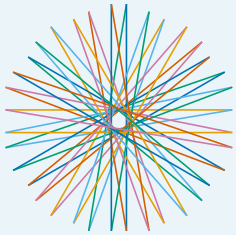
Predict: what will it draw?

Run it. What is the bug, and what is the fix?

Exercise 2.3: Star Burst

Make loop art! Use the same start blocks, then this loop:





What you will see

Now try your own numbers. My favorite: repeat _____ move _____ turn _____

Tip: 20-20-20 for your eyes

Every 20 minutes on a screen, look at something 20 feet (6 meters) away for 20 seconds. Then stretch!

Vocabulary box

loop Code that repeats.

repeat A loop that runs a set number of times, then stops.

forever A loop that keeps going until you click stop.

costume One picture of a sprite. Switching costumes makes animation.

extension Extra blocks you add to Scratch, like the Pen.

degrees How we measure turns. A whole turn is 360 degrees.

Reflection

Where do you see loops in real life?

What was the hardest number to get right today? How did you find it?

Week 3 Level 1: Introduction

When This Happens...

Events start code; conditionals choose what happens.

What you will learn

- How **events** (like a key press or a click) start your scripts.
- How **if** and **if-else** let a sprite choose what to do.
- Why an **if** needs a **forever** loop to keep checking.
- How to give kind feedback: **two stars and a wish**.

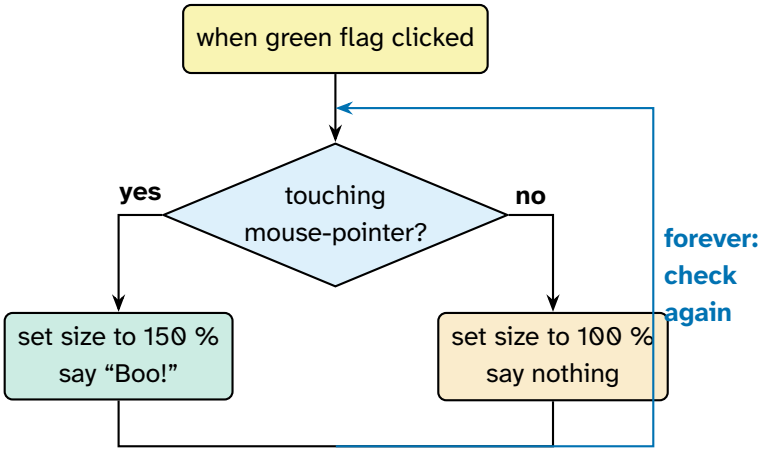
The big idea

Events are doorbells. A script that starts with a yellow **Events** hat block waits quietly. When its doorbell rings (the flag is clicked, a key is pressed, the sprite is clicked), it runs.



Conditionals ask a yes/no question. An **if** block runs the blocks inside it only when the answer is “yes”. An **if-else** block has two mouths: “yes” blocks and “no” blocks. Exactly one of them runs.

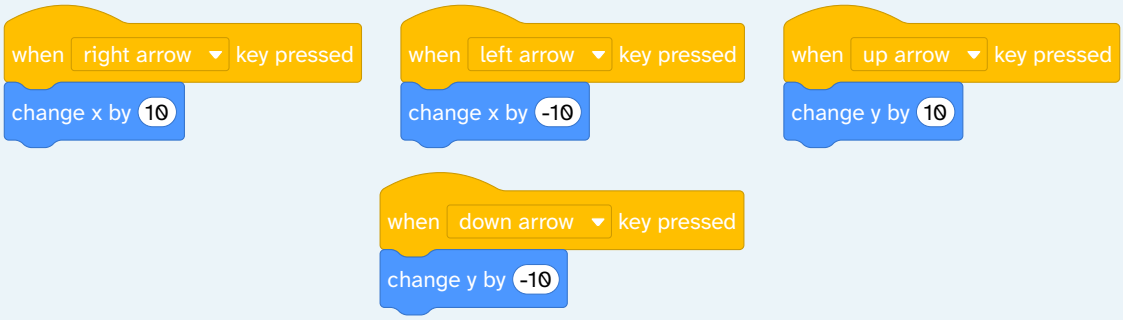
Keep checking! An **if** checks only once, when it runs. To keep watching, put it inside **forever**, like a goalkeeper who never stops watching the ball.



Exercise 3.1: Arrow-Key Explorer

Make a sprite move with the arrow keys. Each key is its own doorbell.

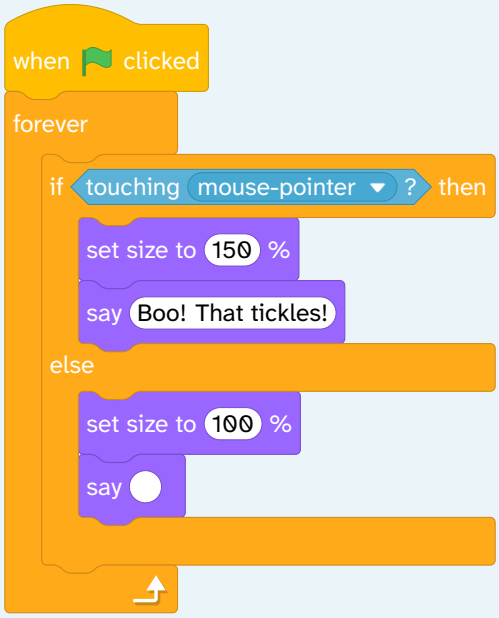
1. New project. From **Events**, drag four "when space key pressed" blocks. Click each dropdown and choose **right arrow**, **left arrow**, **up arrow** and **down arrow**.
2. Under each, snap a **Motion** block: "change x by 10" (right), "change x by -10" (left), "change y by 10" (up), "change y by -10" (down).
3. Click the stage once, then press the arrow keys. Save as `explorer`: you need it for 3.3!



x is left and right. **y** is up and down. Which key makes y bigger? _____

Exercise 3.2: Tickle the Ghost

1. New project. Delete the cat (right-click → **delete**). **Choose a Sprite**: pick **Ghost**.
2. Build the script. The pointy "touching mouse-pointer?" block is in **Sensing**. For the last "say", delete the text so the box is empty.
3. Click the flag, then move your mouse over the ghost and away again.

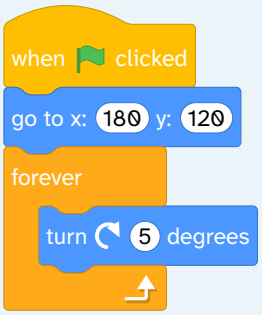


Predict first: what will happen if you take away the “forever”?

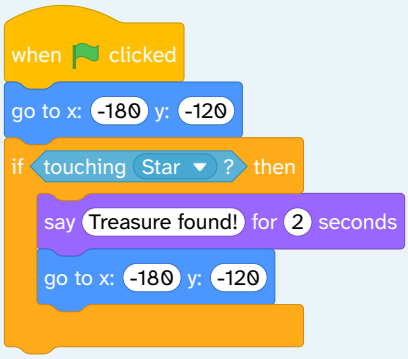
Exercise 3.3: Treasure Hunt

Steer your explorer to the treasure!

1. Open your `explorer` project (**File** → **Load from your computer**).
2. **Choose a Sprite**: add a **Star**. Give the star this script so it waits in the top-right corner and spins:



Bug hunt! Sam gave the explorer this script:



3. Click your explorer sprite and build Sam’s script. Click the flag and steer onto the star. What happens?

4. Why? Fix it! (Remember the goalkeeper.)

Done when: every time the explorer touches the star, it says “Treasure found!” and goes back to the start.

Vocabulary box

event Something that happens (a click, a key press) that starts a script.

condition A yes/no question, like “touching edge?”.

if Runs the blocks inside only when the condition is yes.

if-else Runs one set of blocks for yes and another for no.

sensing Blocks that let a sprite notice things: touching, keys, the mouse.

feedback Comments that help someone improve their work.

Reflection: Two stars and a wish

Show your project to a partner. Write what they said.

★ Star 1:

★ Star 2:

Wish (an idea to make it better):

What is the difference between “when space key pressed” and “if touching…”?

Week 4
Keeping Score
Level 1: Introduction

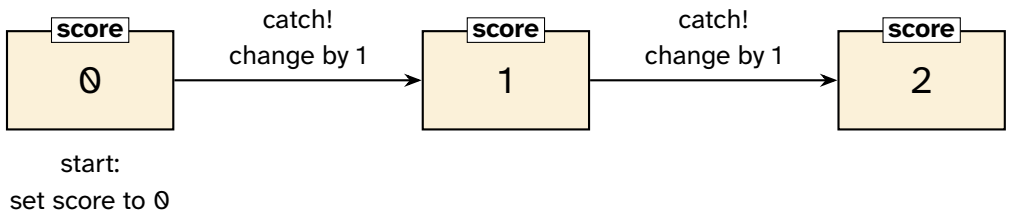
Variables are labeled boxes that remember things.

What you will learn

- How a **variable** remembers a score or lives.
- The difference between **set** and **change**.
- How **pick random** makes your game full of surprises.
- How to build a catch game, and how to give **credit** for art you use.

The big idea

A **variable** is a box with a label. It holds **one** value at a time.

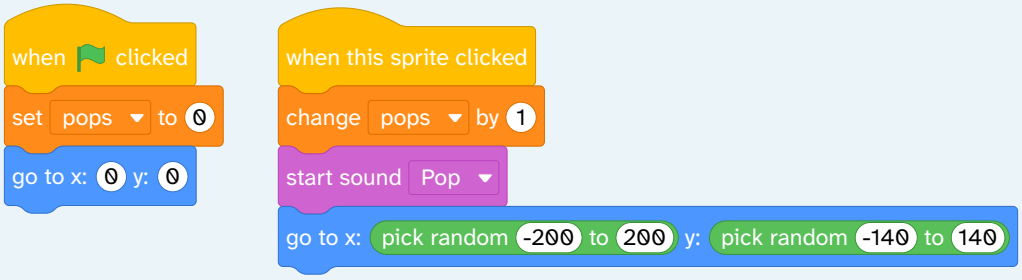


- **set score to 0** empties the box and puts in 0. Use it once, at the start.
- **change score by 1** adds 1 to what is already in the box. Use it during the game.
- **pick random 1 to 6** is like rolling a die: you get a number, but you can't know which!

To make a variable: click **Variables** → **Make a Variable**, type a name, keep "For all sprites", click **OK**. A little score box appears on the stage.

Exercise 4.1: Pop the Balloon

1. New project. Delete the cat. **Choose a Sprite**: pick **Balloon1**.
2. Make a variable called pops.
3. Build both scripts. "pick random" is in **Operators**: drop it into the x and y slots.
4. Click the flag and pop the balloon! Then click the flag again: does pops start at 0?

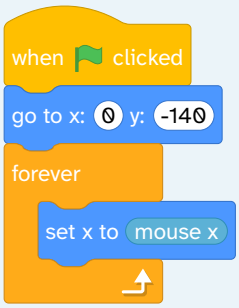


What happens if you take away “set pops to 0”?

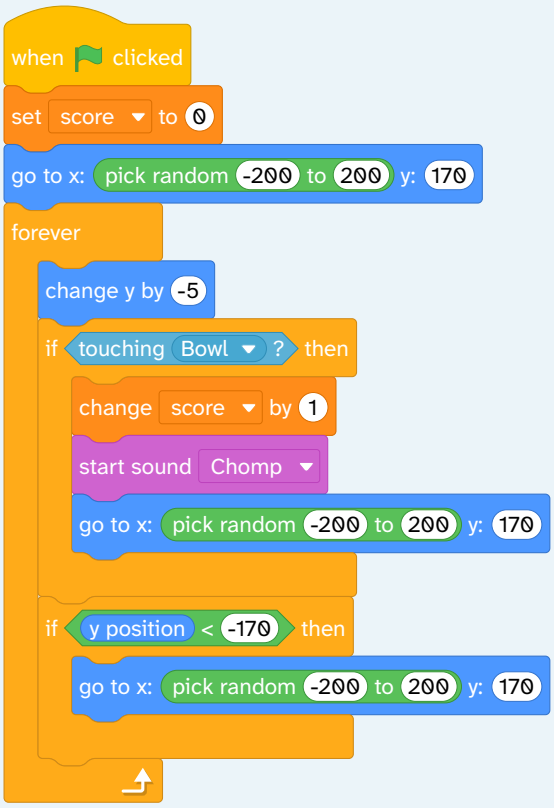
Exercise 4.2: Apple Catch

Catch the falling apples in a bowl!

1. New project. Delete the cat. Add the **Bowl** and the **Apple** sprites. Choose a backdrop you like.
2. Make a variable called `score` (for all sprites).
3. Click the **Bowl** and build its script. “mouse x” is in **Sensing**.
4. Click the **Apple** and build its script. “y position” is in **Motion**; the “<” block is in **Operators**.
5. Play! Move the mouse left and right to catch apples.



Bowl



Apple (pick any sound in the dropdown)

Bug hunt! Mei's apple script is the same, **but** it has no "go to x: (pick random) y: 170" inside the "touching Bowl?" block. Try it: take that block out. What happens to the score when you catch one apple? Why?

Put the block back. **Done when:** each apple you catch adds exactly 1, and missed apples start again at the top.

Exercise 4.3: Lives and Game Over (bonus)

1. Make a variable `lives`. In the Apple's script, under "set score to 0", add "set lives to 3".
2. Inside the "y position < -170" block (a missed apple), add "change lives by -1".
3. At the bottom of the forever loop, add this:



Done when: you start with 3 lives, lose one for each missed apple, and the game stops at 0.

Digital citizenship & safety: Give credit

The Apple, Bowl and Balloon were drawn by real people and shared in the Scratch library for everyone to use. Add a credits note: right-click the code area → **Add Comment**, then type: "Credits: sprites and sounds from the Scratch library." If you change a friend's project, that is a **remix**: say whose project you started from.

Vocabulary box

variable A labeled box that remembers one value, like a score.

set Put a new value in the box (the old one is thrown away).

change Add to the value already in the box (use a minus number to take away).

random A surprise number you cannot predict, like rolling a die.

credit Saying who made the art, sounds or ideas you used.

remix Your own new version of someone else's project, with credit.

Reflection

What is the difference between "set" and "change"?

Whose art did you use today? How did you give credit?

Week 5
Messages and My Blocks
Level 1: Introduction

Break big programs into named parts.

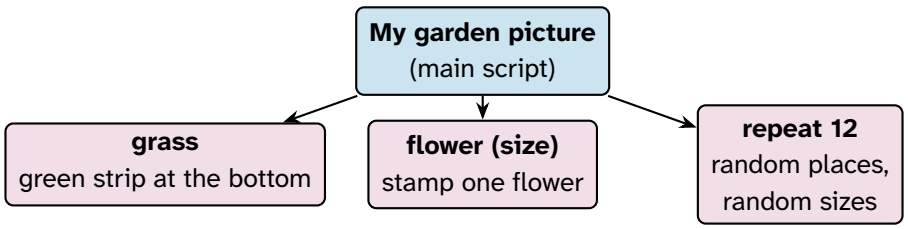
What you will learn

- How to break a big job into smaller named parts (**decomposition**).
- How to make your own block with **My Blocks**, and give it an **input**.
- How sprites send each other **messages** with **broadcast**.
- How to pick colors and text that **everyone** can see.

The big idea

Big jobs, small named parts

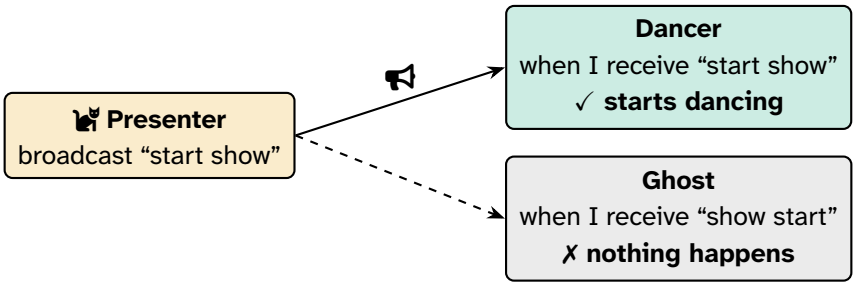
A garden picture is a big job. Break it into parts and give each part a name. Then the main plan is short and easy to read.



In Scratch, **My Blocks** → **Make a Block** lets you make a pink block with your own name. You put its steps under a **define** hat. An **input** is a blank you fill in each time you use the block, like “size”.

Messages

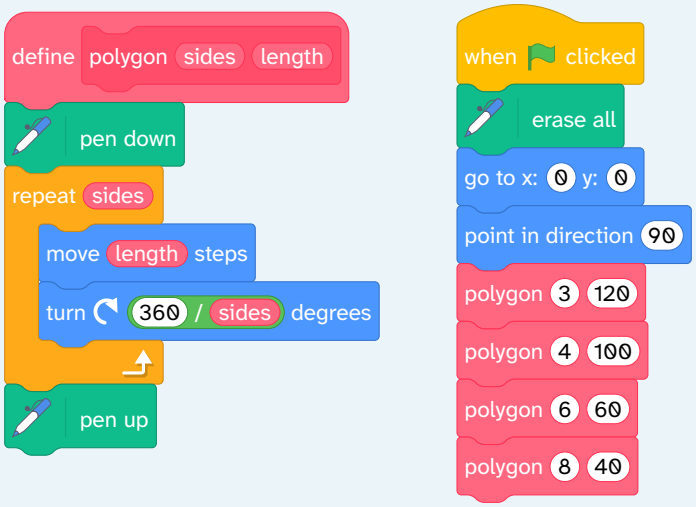
broadcast sends a message to **every** sprite, like an announcement on a loudspeaker. Only scripts that start with **when I receive that exact message** run.



Exercise 5.1: Shape Machine

Make your own block that can draw **any** shape.

1. New project. Add the **Pen** extension. Set the cat’s **Size** to 40.
2. **My Blocks** → **Make a Block**. Type `polygon`. Click **Add an input (number or text)**, type `sides`. Add another input: `length`. Click **OK**.
3. Build the define script. Drag the `sides` and `length` ovals **out of the define hat** into the slots.
4. Build the green-flag script with your new pink block. Click the flag!
5. Add a pentagon: one more block, “polygon 5 70”.

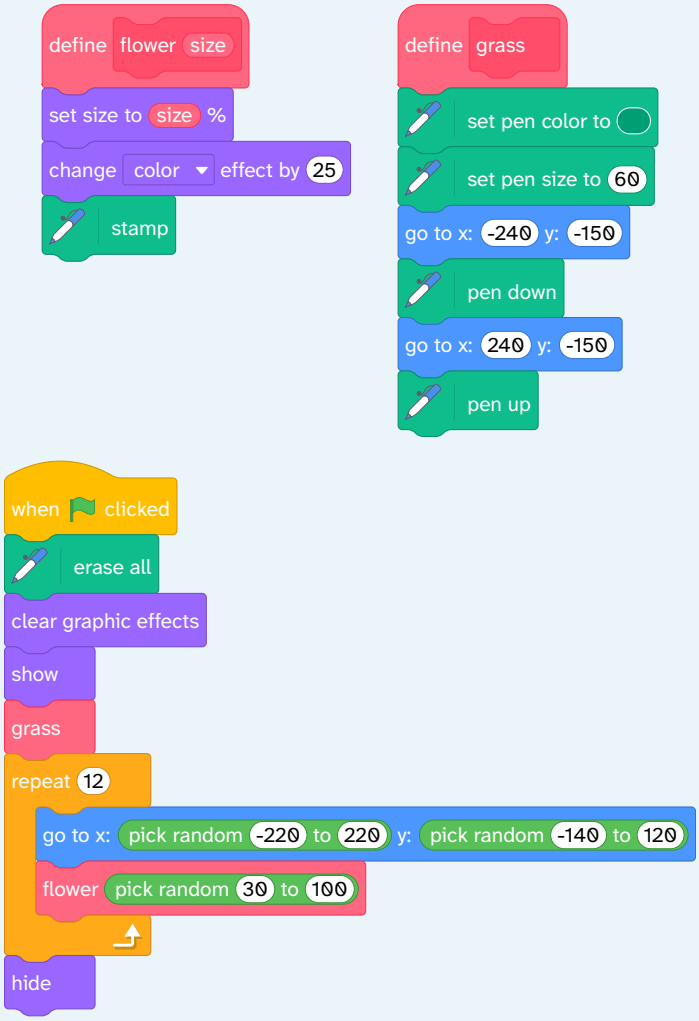


How many blocks would you need to draw these 4 shapes **without** your My Block? _____

Exercise 5.2: Stamp Garden

1. New project with the **Pen** extension. Delete the cat. Hover over **Choose a Sprite** and click the **Paint** brush. Draw a flower in the middle of the canvas (circles for the center and petals). Name the sprite `Flower`.

2. Make two My Blocks: `flower` with an input `size`, and `grass` with no input.
3. Build the three scripts. For the grass color, click the color circle and set **Color 45, Saturation 100, Brightness 62** (see the color-safe paint box).



Done when: each click on the flag grows a new garden: a strip of grass and 12 flowers of different sizes and colors in surprise places.

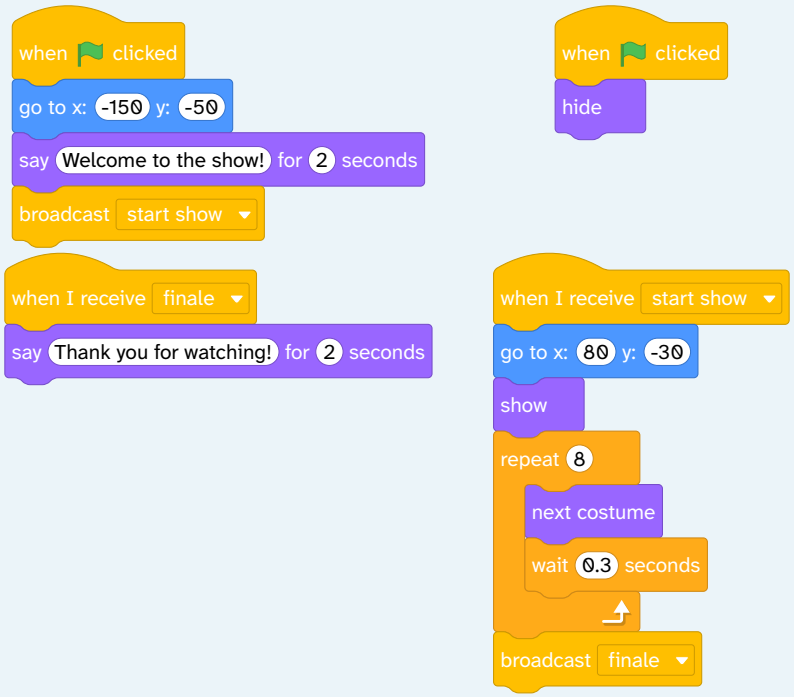
Exercise 5.3: Show Time

Two sprites put on a show by sending messages.

1. New project. Keep the cat (the **Presenter**). Add a **Dancer**: any sprite with several costumes, like Ballerina.
2. To make a message: in a “broadcast” block, click the dropdown → **New message**, type `start`

`show`. Make another called `finale`.

3. Build the scripts. Click the flag and enjoy the show!



Presenter (cat) **Dancer**

Bug hunt! Ada's dancer starts with "when I receive **show start**". Try it. What happens? Why? How do you fix it?

Digital citizenship & safety: Color-safe paint box

Some people see colors differently. These colors stay different for almost everyone. In Scratch, click a color circle and type the three numbers. And never use color as the only clue: add a shape, a pattern or a word too, and keep text big.

Color	Col.	Sat.	Bri.	Color	Col.	Sat.	Bri.
Orange	12	100	90	Blue	56	100	70
Sky blue	56	63	91	Vermillion	7	100	84
Bluish green	45	100	62	Reddish purple	91	41	80
Yellow	16	72	94	Black	0	0	0

Vocabulary box

decomposition Breaking a big problem into smaller parts.

My Block A block you make yourself, with a name you choose.

define The hat block that holds the steps of your My Block.

input A blank in a block that you fill in each time, like “size”.

broadcast Send a message to every sprite.

when I receive A hat block that runs when its exact message is sent.

accessible Easy for everyone to use, including people who see or hear differently.

Reflection

Why is “polygon 6 60” easier to read than the blocks inside it?

What did you do to make your project easier for everyone to see?

Week 6
Review Studio and Checkpoint
Level 1: Introduction

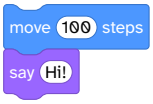
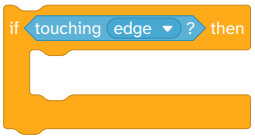
Show what you know; get ready for text code.

What you will learn

- How Scratch blocks turn into **Python** words (next week's language!).
- How to be a bug detective in projects someone else made.
- What makes a project **fun** and **fair** for everyone.

The big idea: our six big ideas

Every programming language in the world uses these six ideas. You know them all!

Big idea	It means...	In Scratch
Sequence (week 1)	Steps happen in order, one after another.	
Loops (week 2)	Repeat steps so you don't copy them.	
Events (week 3)	A doorbell starts a script.	
Conditionals (week 3)	Ask a yes/no question, then choose.	
Variables (week 4)	A labeled box that remembers.	
My Blocks (week 5)	Name a part and use it again.	

Tip: The four detective steps

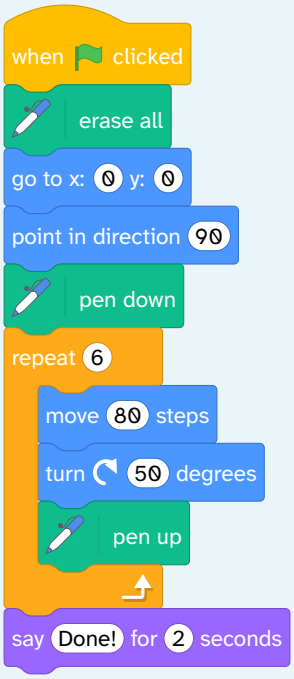
1. What did I **expect**? **2.** What **happened**? **3.** Read the blocks one at a time with your finger. **4.** Change **one** thing, then test again.

Bug-hunt relay

Each project card shows a project that someone built with **two bugs**. Build it **exactly** as shown (bugs and all!), click the flag, then be detectives. Fill in the relay baton for each one.

Exercise 6.1: Relay A: Wonky Hexagon

It should: draw one closed hexagon (6 equal sides of 80 steps) in the middle of the stage, then the cat says “Done!”.
Build it: new project, add the **Pen** extension, set the cat’s **Size** to 40, then build this script on the cat:



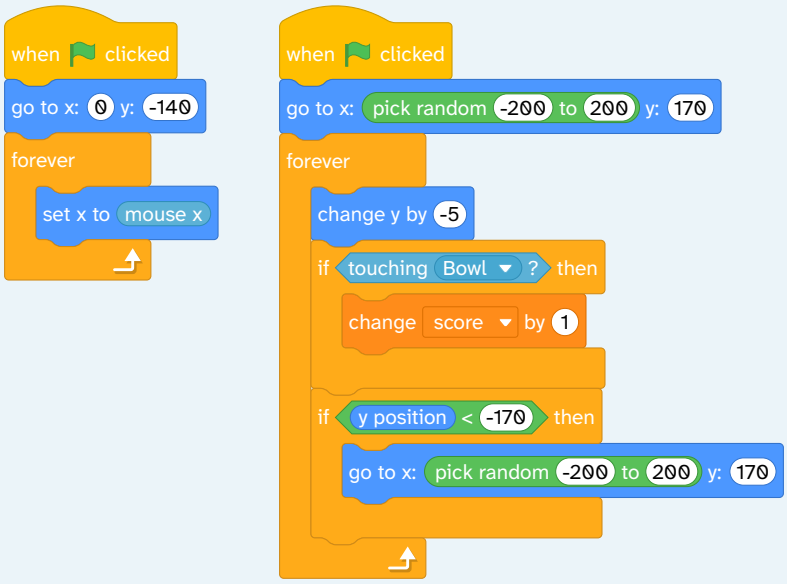
Relay baton. What happened?

Bug 1 and fix:

Bug 2 and fix:

Exercise 6.2: Relay B: Apple Catch Can't Count

It should: start every game with a score of 0; drop apples from surprise places at the top; add **exactly 1** for every apple caught in the bowl; send missed apples back to the top.
Build it: new project, delete the cat, add the **Bowl** and **Apple** sprites, make a variable `score` (for all sprites), then:



Bowl **Apple**

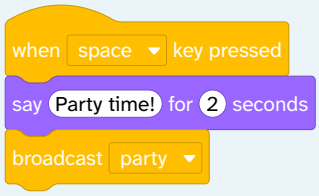
Play two games. **Relay baton.** What happened?

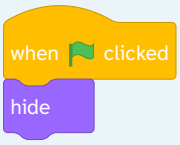
Bug 1 and fix:

Bug 2 and fix:

Exercise 6.3: Relay C: The Silent Party

It should: after the green flag, the Ghost hides. When you press `space`, the cat says "Party time!", then the Ghost appears, does a slow spinning dance you can really watch (one full turn, about 2 seconds) and says "Woo!".
Build it: new project, keep the cat, add the **Ghost**. Make two messages called `party` and `dance` (broadcast dropdown → **New message**). Then:







Cat **Ghost**

Relay baton. What happened?

Bug 1 and fix:

Bug 2 and fix:

Blocks to words

Next week you will write code in **Python**. Python uses words instead of blocks. Match each block card (A–L) with its Python card (1–12). Write the number next to each letter.

A	B	C	D	E	F	G	H	I	J	K	L

What do you notice about Python? (Look at the brackets, quotes and colons.)

Vocabulary box

debugging Finding and fixing bugs, step by step, like a detective.

Python A programming language where you type words instead of snapping blocks.

text code Code made of typed words and symbols.

checkpoint A short check that shows what you know and what to practice next.

fair Same rules for everyone, no tricks, and everyone can play.

Reflection: Fun and fair for everyone

One thing that makes a game **fun**:

One thing that makes a game **fair**:

The project I am proudest of from weeks 1–6, and why:

Printable: block cards (cut along the lines)

A



B



C



D



E



F



G



H



I



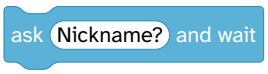
J



K



L



Printable: Python cards (cut along the lines)

1 `t.pendown()`	**2** `score = 0`	**3** `for i in range(4):`
4 `roll = random.randint(1, 6)`	**5** `t.right(90)`	**6** `print("Hello!")`
7 `if score > 10:`	**8** `t.goto(0, 0)`	**9** `name = input( "Nickname? ")`
10 `t.penup()`	**11** `score = score + 1`	**12** `t.forward(100)`

Week 7 Level 1: Introduction

Hello, Python!

Text code is just blocks written as words — and Python is picky.

What you will learn

- How to open, run and save a program in **Thonny**.
- How to make Python say things with `print`, and leave notes with comments.
- How to make the turtle draw a square with `forward` and `left`.
- How to read a red error message like a detective.

Warm-up: Spot the difference

Python is very picky. On each card, one line works and one line confuses Python. Circle the difference. Check the line that works.

Card	Line A	Line B
1	☐ `print("Hello")`	☐ `Print("Hello")`
2	☐ `t.forwad(100)`	☐ `t.forward(100)`
3	☐ `t.left(90`	☐ `t.left(90)`
4	☐ `t.left(90)`	☐ `t.Left(90)`
5	☐ `print(Hi there)`	☐ `print("Hi there")`

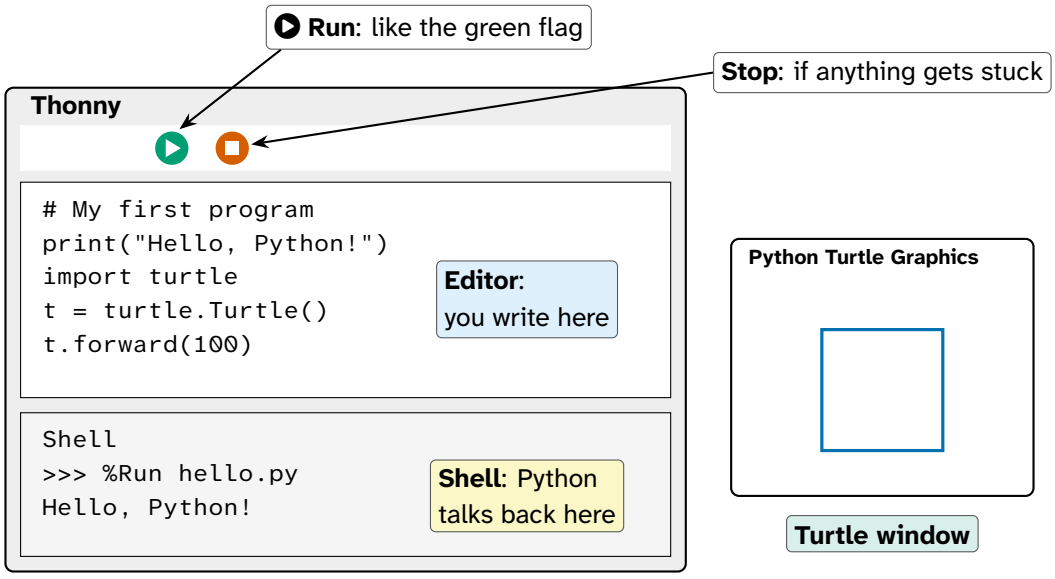
The big idea: blocks become words

In Scratch you snapped blocks together. In Python you **type** the same ideas as words. The computer still does the lines in order, from top to bottom.

Scratch block		Python line
when clicked	→	the green **Run** button (or F5)
move 100 steps	→	`t.forward(100)`
turn 90 degrees	→	`t.left(90)`
say Hello!	→	`print("Hello!")`

Meet Thonny

Thonny is the app where we write Python. It has three important parts.



Tip: Where did my turtle go?

The turtle window sometimes hides behind Thonny. Click the Python icon in the Dock at the bottom of the screen.

Every turtle program has the same shape

```
import turtle              # 1. get the turtle toolbox
t = turtle.Turtle()        # 2. make a turtle and call it t
t.forward(100)             # 3. your drawing commands go here
t.left(90)
turtle.done()              # 4. keep the window open at the end
```

Lines that start with # are **comments**. Python skips them. They are notes for people.

Exercise 7.1: Hello, Python!

Folder: `week07/ex-hello-python`. Open `starter.py` and press Run.

1. Add three `print` lines: who is coding today (a **nickname**), your favorite game or animal, and a goodbye that uses the word `bye`.
2. Press Run after every new line.

Done looks like: four different lines in the Shell. Check: open `check_my_work.py` and press Run.

Exercise 7.2: My First Square

Folder: `week07/ex-first-square`. The turtle draws one side already. Add the other three sides: each side is `t.forward(100)` then `t.left(90)`. Done looks like: a closed square — the turtle ends where it started. Then run `check_my_work.py`.

Reading a red error message

When Python is confused, it stops and shows red text in the Shell. Don't panic — it is a clue!

Output

```
Traceback (most recent call last):
  File "starter.py", line 12, in <module>
    Print("Typo hunt: fixed!")
NameError: name 'Print' is not defined
```

- **line 12** tells you **where** to look.
- The **last line** tells you **what** confused Python: it does not know the word `Print`.

Exercise 7.3: Typo Hunt

Folder: `week07/ex-typo-hunt`. This program should draw a flag on a pole, but it has **4 typos**. Run it, read the red message, fix **one** typo, run again. Write down what you fixed:

Line	The typo was	I changed it to

Tip: Save your work safely

Exercises: press Cmd+S to save `starter.py` in its folder. New programs go in `Documents/my-python`. Use names that say what the program does, like `flag-drawing.py`. Never call a file `turtle.py`: Python would get mixed up with the real turtle!

Vocabulary box

Python A text programming language. We type words instead of snapping blocks.

Thonny The app where we write and run Python programs.

Editor The top part of Thonny where you write your program.

Shell The bottom part of Thonny where Python shows messages and errors.

print A command that shows words in the Shell: `print("Hi")`.

Comment A note for people that starts with #. Python skips it.

Syntax error A typo or missing bracket that stops Python from understanding your code.

Bug A mistake in a program. Every programmer makes them!

Reflection

What was different about typing code instead of dragging blocks?

Which typo was the hardest to spot? What will you check first next time?

Week 8 Level 1: Introduction

Loops in Python

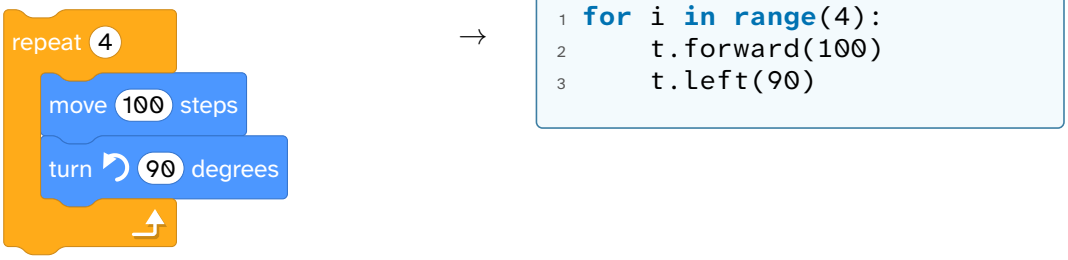
A `for` loop plus pushed-in lines is Scratch's repeat block, written in words.

What you will learn

- How to repeat lines with `for i in range(4):`.
- Why the spaces at the start of a line matter (indentation).
- The secret rule for drawing any shape with equal sides.
- How to put a loop inside a loop to make patterns.

The big idea: repeat, in words

Last week your square needed 8 lines. With a loop it needs 3. Compare:



Python has no C-shaped block. Instead it uses two clues:

1. a **colon** `:` at the end of the `for` line, and
2. **4 spaces** at the start of every line **inside** the loop. This is called **indentation**.

```
for i in range(3):
    t.forward(100)
    t.left(120)
print("Done!")
```

4 spaces

inside the loop: repeats 3 times

after the loop: runs once

Tip: Thonny helps you

After a colon, press `Return`: Thonny pushes the next line in for you. To come back out of the loop, press `Delete` once.

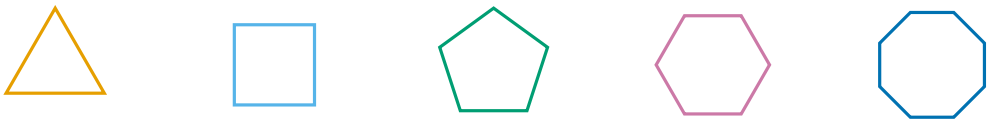
How many times?

`range(4)` gives Python four numbers: `0`, `1`, `2`, `3`. Python starts counting at `0`! What matters is that there are **4** of them, so the loop runs 4 times.

The secret shape rule

To draw a shape and get back to the start, the turtle turns all the way round: `360` degrees. Share that turn between the corners:

turn at each corner = 360 ÷ number of sides



triangle	square	pentagon	hexagon	octagon
3 sides	4 sides	5 sides	6 sides	8 sides
turn 120	turn 90	turn ?	turn ?	turn ?

Fill in the missing turns: pentagon _____, hexagon _____, octagon _____.

Exercise 8.1: Loop Square

Folder: `week08/ex-loop-square`. Run the starter: the turtle walks in a straight line! Add `t.left(90)` **inside** the loop, lined up under `t.forward(100)`. Run it, then open `check_my_work.py` and press Run.

Exercise 8.2: Polygon Party

Folder: `week08/ex-polygon-party`. The triangle is done. Write a loop for a **square** (sides of 100) and a loop for a **hexagon** (sides of 60). Use the shape rule for the turns. Done looks like: an orange triangle, a sky-blue square and a green hexagon, all closed.

A loop inside a loop

A loop can hold another loop. The **outer** loop picks a color. The **inner** loop draws a whole square. Then the turtle turns a little and the outer loop goes again.

```
for color in PALETTE:          # 8 colors: runs 8 times
    t.color(color)
    for side in range(4):       # draws one whole square
        t.forward(100)
        t.left(90)
    t.left(45)                  # 8 x 45 = 360
```

The inner lines have **8** spaces: 4 for the outer loop and 4 more for the inner loop. How many lines does the turtle draw altogether? _____

Exercise 8.3: Square Flower

Folder: `week08/ex-square-flower`. The starter draws one square. Put the square loop inside a color loop (like the code above) to make 8 squares in 8 colors. Tip: `t.speed(0)` draws super fast.

Tip: Asking for help well

Stuck? First try "three before me": read the red message, check this workbook, ask your partner. Then ask a grown-up with the help sentence:
I expected _____, but _____ happened. I tried _____.

Vocabulary box

Loop Code that repeats.

for The Python word that starts a loop: `for i in range(4):`.

range Gives the loop its numbers. `range(4)` means 0, 1, 2, 3: four times.

Indentation The spaces at the start of a line. They show which lines are inside the loop.

Nested loop A loop inside another loop.

Off-by-one bug When a loop runs one time too many or one time too few.

Reflection

How is `for i in range(4):` like Scratch's repeat block? How is it different?

Draw (or describe) the coolest pattern you made today:

Week 9 Level 1: Introduction

Variables, Input and Choices

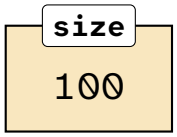
Programs can ask questions, remember the answers and decide what to do.

What you will learn

- How a **variable** remembers a value.
- How to ask a question with `input`, and turn the answer into a number with `int`.
- How `if`, `elif` and `else` let a program choose.
- Which questions a program should never ask.

The big idea: labeled boxes

Remember the `score` variable in Scratch? A variable is a box with a label. You put something in it, and you can use it later.

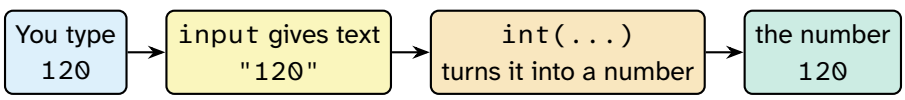


`size = 100` puts `100` in the box labeled `size`.
`t.forward(size)` takes it out and uses it: `100` steps.

One equals sign = means **put in the box**. (Two equals signs == mean something else — see below!)

Asking a question

`input` asks a question in the Shell and waits. Click in the Shell, type your answer and press `Return`. But watch out: `input` **always** gives back **text**, even when you type a number.



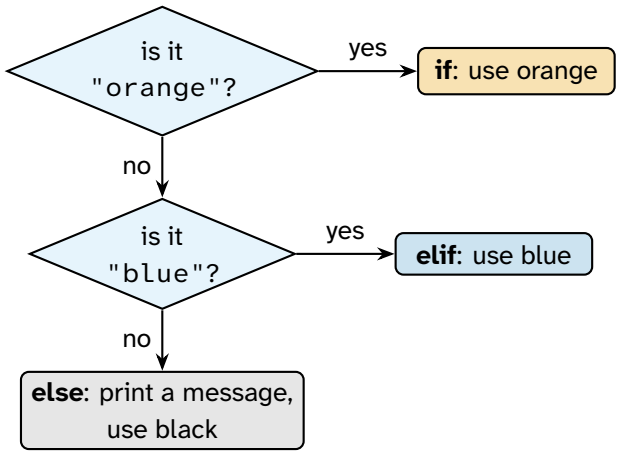

```
answer = input("How big should the square be? ")
size = int(answer)
```

Exercise 9.1: Square of Any Size

Folder: `week09/ex-size-it`. Run the starter and type `120` in the Shell. A red `TypeError`! The turtle cannot walk the text `"120"`. Fix it with `size = int(answer)`. Try 50, 150 and 250.

Making choices: if, elif, else

A program can choose what to do. Python checks the rules from the top. The **first** rule that is true wins, and the others are skipped. `else` catches everything that is left.




```
if color_name == "orange":
    color = "#E69F00"
elif color_name == "blue":
    color = "#0072B2"
else:
    print("I do not know that color. I will use black.")
    color = "#000000"
```

Every `if`, `elif` and `else` line ends with a colon, and the lines under it are pushed in 4 spaces — just like loops.

Comparing

Sign	Means	Example that is true
`==`	is the same as	`"blue" == "blue"`
`!=`	is not the same as	`"Blue" != "blue"` (Python is picky!)
`<`	is less than	`3 < 10`
`>`	is more than	`25 > 10`

Exercise 9.2: Pick a Color

Folder: `week09/ex-color-choice`. The program knows orange and blue. Add an `elif` for `"green"` (`"#009E73"`) and an `else` that prints a friendly message and uses black. Test with `orange`, `blue`, `green` and `pizza`. The color name is written under the circle, so you can tell the colors apart even without seeing color.

Exercise 9.3: How Many Stars?

Folder: `week09/ex-star-row`. Ask how many stars (1 to 10). Before drawing: **if** the number is more than 10, print a message and use 10; **elif** it is less than 1, print a message and use 1; **else** print `"Coming right up!"`. Test with 3, 25 and 0.

Questions a program should NOT ask

Check one box for each question.

Question	OK	Never
What is your favorite color?	☐	☐
Where do you live?	☐	☐
How many stars should I draw?	☐	☐
What is your password?	☐	☐
What is your hero name?	☐	☐
What school do you go to?	☐	☐
What is your full name?	☐	☐
What is your favorite animal?	☐	☐

Our rule: ask about **choices** (colors, sizes, numbers), never about **who you are**. If an app or game asks for personal details, stop and ask a grown-up.

Vocabulary box

Variable A labeled box that remembers a value, like `size = 100`.

input Asks a question in the Shell and gives back the answer as text.

int Turns text like `"120"` into the number 120.

Text (string) Letters or numbers inside quote marks, like `"blue"` or `"120"`.

if / elif / else Python's way to choose: "if this, else if that, otherwise..."

Comparison A question with a yes/no answer, like `stars > 10`.

Reflection

Think of a game you know. Where does it ask you something and then decide what happens?

Why does `input` need `int` for sizes, but not for color names?

Week 10 Level 1: Introduction

Random Art and My Own Commands

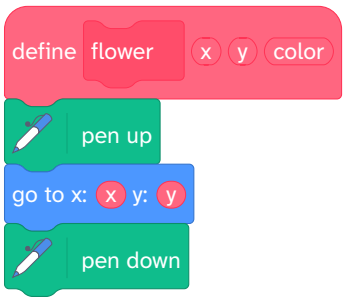
`random` makes surprises; `def` makes your own command, just like My Blocks.

What you will learn

- How to make your own command with `def`, with input slots called **parameters**.
- The difference between **defining** a command and **calling** (using) it.
- How `random` makes a new picture every time you press Run.
- How to plan your final project: **My Interactive Art Show**.

The big idea: make your own command

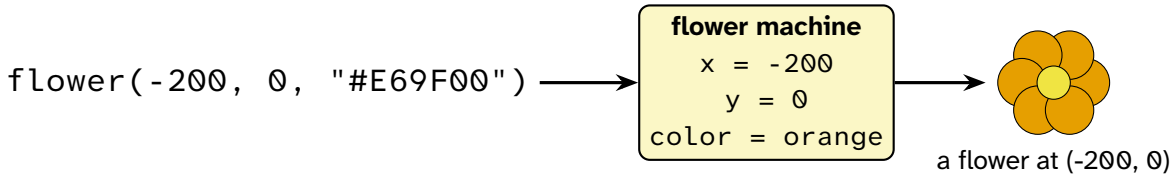
In Scratch you made My Blocks. Python has them too! You write `def`, a name, and the input slots in brackets.



→

```
def flower(x, y, color):
    t.penup()
    t.goto(x, y)
    t.pendown()
    t.color(color)
    ...
```

Defining writes the recipe. Nothing is drawn yet! **Calling** uses the recipe. The values in the brackets go into the slots, in order:



Tip: Remember

Define first (at the top), call later (at the bottom). Calls start at the left edge — not pushed in — or they become part of the recipe.

Exercise 10.1: Flower Stamp

Folder: `week10/ex-flower-stamp`. Run the starter: nothing is drawn! The recipe is there, but nobody calls it.

1. At the bottom, call it three times: `flower(-200, 0)`, `flower(0, 0)`, `flower(200, 0)`.
2. Add a third parameter: `def flower(x, y, color):` and use `t.color(color)`.
3. Give each call a color: `flower(-200, 0, PALETTE[0])`, and so on.

Done looks like: three flowers in different colors, each with a yellow middle. Then run `check_my_work.py`.

Surprise! The random module

`random.randint(1, 6)`	Like rolling a dice: a whole number from 1 to 6.
`random.randint(-350, 350)`	A random spot from left (-350) to right (350).
`random.choice(PALETTE)`	Like picking a crayon from a bag with your eyes closed.

Put `import random` at the top of your program, next to `import turtle`.

Exercise 10.2: Random Bubble Art

Folder: `week10/ex-random-art`. The starter draws 30 bubbles... all in the same place! Inside the loop, change the fixed numbers to random ones: `x`, `y`, `size` with `random.randint`, and `color` with `random.choice(PALETTE)`. Press Run three times. Is it different each time?

Your final project: My Interactive Art Show

For weeks 10 to 12 you will make your own program for an art show. It says hello, asks the viewer one or two questions, and draws a picture that changes because of the answers. It must use these **six ingredients**:

☐ a question with `input`

- ☐ a variable that remembers an answer or a number
- ☐ an `if` that chooses what to draw
- ☐ a `for` loop that repeats something
- ☐ `random` for a surprise
- ☐ a function you made with `def`

Ideas: a fish party under the sea, a garden of flowers, a city skyline at night, space with planets, a birthday card, a pattern machine. Write three ideas you like:

Then fill in the **plan sheet** in your Project Brief (Milestone M1): a sketch, your questions, and your function's name and inputs.

Tip: Be an original artist

Borrowing an idea is fine when you make it your own and say thank you. Write a comment like `# Flower idea remixed from Exercise 10.1`. Questions in your project ask about choices — colors, numbers, day or night — never about who someone is.

Vocabulary box

Function A command you make yourself, with a name. Like a My Block.

def The Python word that starts a function's recipe: `def flower(x, y, color):`.

Parameter An input slot of a function, like `x`, `y` or `color`.

Call Using a function: `flower(0, 0, "#CC79A7")`.

random A Python toolbox for surprises: `randint` and `choice`.

Remix Making something new from someone's idea, and giving them credit.

Reflection

What command would you like to invent for your project? What would its input slots be?

Why do you think a function is useful when you need ten flowers?

Week 11
Project Studio
Level 1: Introduction

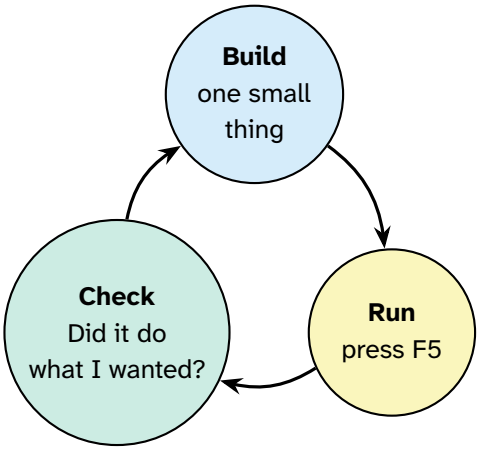
Build, test and improve your project in small steps.

What you will learn

- How real programmers build big programs: one small step at a time.
- How to test a partner's program and give helpful feedback.
- How to finish Milestones M2 and M3 of your art show.

The big idea: build, run, check

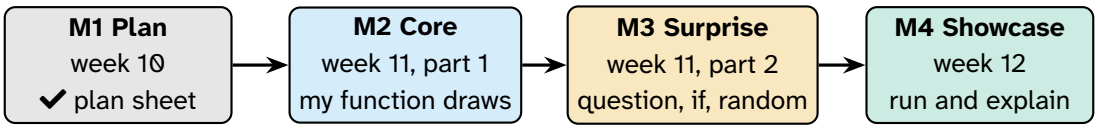
Big programs are not written in one go. Programmers go round this loop all day long:



If something breaks, you know it was the **last small change**.

It is like building a LEGO set bag by bag: finish bag 1, check it looks like the picture, then open bag 2.

Your milestones



Five small steps (the tiny art show from the demo)

1. **Question**: `how_many = int(input("How many bubbles? "))`. Run.
2. **Function**: `def bubble(x, y, color):` and call it once. Run.
3. **Loop**: put the call inside `for i in range(how_many):`. Run.
4. **Random**: random x, y and color. Run twice!
5. **If**: if the number is too big, use a smaller one. Run with `500`.

Tip: Save a version

Before a big change, click **File** → **Save as...** and add `_v2` to the name (`fish_party_v2.py`). If the change goes wrong, version 1 is still safe.

Exercise 11.1: Sprint 1: core drawing works (M2)

Open your project file. Change the example `my_shape` function into **your** function: give it a name that says what it draws, and change the drawing. Draw it once at `(0, 0)`. Run. Then call it a second time somewhere else. Run.

- ☐ My function has a good name: _____
- ☐ It draws my main thing.
- ☐ I call it at least twice, in different places.
- ☐ The program runs with no red errors.

Exercise 11.2: Partner bug hunt

Swap seats with your partner. Run their program **twice**: once with a normal answer, once with a strange one (`0`, a giant number, a word, a capital letter). Write what happened — don't fix it, just report it!

I typed	What happened

Then give **two stars and a wish**:

★ I liked _____

★ I liked _____

I wish _____

Exercise 11.3: Sprint 2: surprise and polish (M3)

Add the rest of your six ingredients. Check off each one in the comment list at the top of your file **when you have seen it work**.

- ☐ My question(s) work (numbers use `int(...)`).
- ☐ My `if` changes something you can see.
- ☐ My loop repeats my function.
- ☐ `random` makes it different every run.
- ☐ A title with `t.write(...)` and my **nickname** as the artist.

Tip: Feedback is a gift

Giving: be kind and specific ("I like how the stars are different sizes"), and make wishes about the program, never the person. **Getting**: say thank you — then **you** decide what to change. It is your art.

Vocabulary box

Iterate To improve something by going round the build–run–check loop again and again.

Milestone A checkpoint on the way to finishing a project.

Playtest Letting someone else try your program to find problems.

Feedback What someone tells you about your work, to help you improve it.

Version A saved copy of your program at one moment, like `_v2`.

Reflection

What bug did you find today? How did you find it?

What feedback did you get? What will you change because of it?

Week 12 Level 1: Introduction

Showcase and Celebration

Share what you made, explain how it works, and look back at how far you have come.

What you will do today

- Show your art show to visitors and explain one part of your code.
- Visit the other artists and give kind, helpful feedback.
- Do the exit assessment: a short quiz to show what you have learned.
- Celebrate! You are a Level 1 coder.

The big idea: tell the story of one part

A museum guide doesn't explain every brush stroke. They pick one interesting thing and tell its story. You will do the same: pick **one** part of your code — your function, your loop or your `if` — and explain it in your own words.

Exercise 12.1: Prepare your 2-minute demo

Fill in the gaps. Practice once with your partner: the **artist** talks and points, the **assistant** types the visitor's answer and presses Run. Then swap.

1. "Hi! My project is called _____. It draws _____."
2. "It will ask you a question. Do you want to answer it?" (Press Run.)
3. "Look: because you said _____, it drew _____. That is my `if`."
4. "This is my function _____. It draws _____. Here is where I use it inside a loop."
5. "One bug I fixed was _____."
6. "Do you have a question for me?"

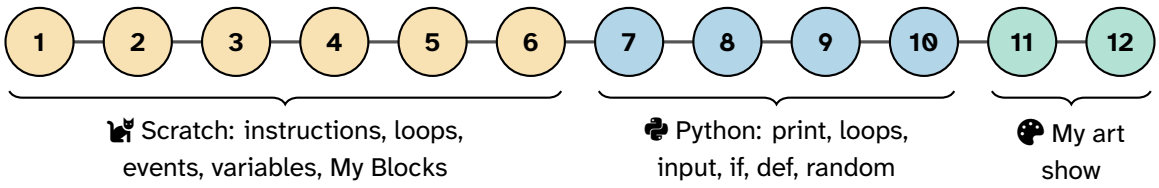
Tip: Showcase safety

Projects are shown on the screen only: nothing is uploaded, posted or emailed. Name cards and screens show **nicknames** only. Photos of your artwork only if you agree, with no faces or names on the screen. If you borrowed an idea, say so in your demo.

Exercise 12.2: Be a great visitor

At each stand: answer the program's question, ask the artist one question, and fill in a feedback card: **two stars** (two things you liked) and **a wish** (one idea to make it even better). Write here the best idea you saw today:

Look back: your Level 1 journey



Exercise 12.3: What I can do now

Check off each sentence that is true for you. Be proud of every tick!

- ☐ I can write exact step-by-step instructions in the right order.
- ☐ I can use a loop to repeat things and make patterns.
- ☐ I can make a program react with `if`, `elif` and `else`.
- ☐ I can use variables, `input` and `random`.
- ☐ I can make my own command with `def`.
- ☐ I can find and fix a bug by reading the error message.
- ☐ I can work as a driver and as a navigator, and explain my code.
- ☐ I keep my personal information private and give credit for ideas.

Tip: Coming up in Level 2

Lists of things, games that react to keys and timers, programs that remember high scores in files — and you will even write your own checks, like the `check_my_work` programs you used this term.

Vocabulary box

Showcase An event where you show your work to others.

Demo Short for demonstration: running your program and showing how it works.

Feedback card A card with two stars and a wish for an artist.

Reflection Thinking back about what you did, what you learned and what you want next.

Reflection

What part of your art show are you most proud of?

What was the hardest bug you fixed this term? How did you fix it?

What do you want to make next?